

Au-Park Project Overview

Au-Park는 Flutter 모바일 앱, FastAPI 백엔드, OpenCV 기반 비전 워커를 연결해 차량 등록, 소셜 로그인, 번호판 인식 입출차, 사전정산, 자동정산, 주차 현황을 처리하는 스마트 주차 시스템입니다.

작성일: 2026-06-09

Repository: Au-Park-project

Mobile + API + Vision Worker

1. 프로젝트 한눈에 보기

Flutter 모바일 앱

사용자 로그인, 소셜 로그인, 차량 등록, 결제수단 등록, 홈 현황판, 주차 현황, 주차 내역 화면을 담당합니다.

Flutter

Dart

Provider

http

FastAPI 서버

인증, 사용자, 차량, 결제, 주차 세션, LPR 입출차, 요금 정책, Firebase 저장소 연동을 처리합니다.

FastAPI

Pydantic

Firebase RTDB

Uvicorn

Vision Worker

iPhone Continuity Camera 또는 RTSP/IP 카메라 영상을 읽고, 번호판 OCR과 주차면 감지를 서버 이벤트로 전송합니다.

OpenCV

EasyOCR

NumPy

Cloudflare Tunnel

2. 디렉터리 구조

```
Au-Park-project/
├── apps/
│   ├── mobile_flutter/      Flutter iOS/Android 앱
│   └── admin_dashboard/     관리자 대시보드 예정 영역
├── services/
│   ├── api/                 FastAPI 백엔드 서버
│   │   └── app/
│   │       ├── api/         auth, users, vehicles, parking, payments, logs, lpr 라우터
│   │       ├── core/        환경 설정
│   │       ├── schemas/     Pydantic 요청/응답 모델
│   │       └── services/     비즈니스 로직 및 저장소
│   └── tests/               API 테스트
├── vision_worker/           OpenCV/EasyOCR 기반 카메라 워커
│   ├── vision_worker/       camera, plate, occupancy, ocr, api_client
│   └── config.local.yaml     로컬 카메라/API 설정
├── scripts/
│   └── run_mobile_cloudflare.sh Cloudflare Tunnel + Flutter 릴리스 실행 헬퍼
└── project_overview.html     현재 문서
```

3. 개발 스택

영역	기술	역할
모바일 앱	Flutter, Dart, Provider, SharedPreferences, http	사용자 화면, 상태 관리, API 통신, 로컬 사용자 정보 보관
소셜 인증	google_sign_in, Kakao Flutter SDK, flutter_naver_login	Google ID Token, Kakao/Naver Access Token 획득 후 서버 검증
백엔드	FastAPI, Uvicorn, Pydantic Settings, httpx	REST API, 인증, 주차 세션, 결제, LPR 이벤트 처리
데이터 저장소	Firebase Realtime Database, InMemory Repository	운영/테스트 데이터 저장. 환경변수 없을 때는 인 메모리 저장소 사용
비전 처리	OpenCV, EasyOCR, NumPy, PyYAML	카메라 프레임 수집, 번호판 후보 검출, OCR, 주차면 상태 감지

영역	기술	역할
무선 테스트	Cloudflare Tunnel	MacBook의 로컬 FastAPI 서버를 HTTPS 공개 주소로 노출
결제	iamport_flutter, 서버 결제 내역 모델	결제수단 등록 화면, 사전정산/자동정산 결제 내역 생성

4. 전체 아키텍처

1 Flutter 앱

사용자가 로그인, 차량 등록, 결제수단 등록을 수행합니다. 앱은 `ApiClient` 를 통해 FastAPI 와 통신합니다.

2 FastAPI 서버

사용자, 차량, 결제수단, 주차 세션, 결제 내역을 Firebase 또는 인메모리 저장소에 기록합니다.

3 Vision Worker

iPhone Continuity Camera 또는 RTSP/IP 카메라에서 프레임을 읽고 번호판을 OCR로 인식합니다.

4 LPR 자동 입출차

`/api/v1/lpr/auto` 는 활성 주차 세션이 없으면 입차, 있으면 출차로 처리합니다.

5 홈 현황판 갱신

앱 홈 화면은 `/api/v1/parking/sessions/active` 를 주기적으로 조회해 차량번호, 입차 시간, 기본요금, 추가요금, 정산완료, 미정산 금액을 표시합니다.

5. 주요 기능

인증 및 사용자

- 이메일 로그인/회원가입
- Google, Kakao, Naver 소셜 로그인
- 서버에서 소셜 토큰 검증 후 Au-Park 사용자 생성/조회

차량 및 결제수단

- 사용자별 차량번호 등록
- 결제수단 등록 화면
- 사전정산, 자동 출차 정산, 추가요금 정산 내역 저장

LPR 입출차

- 번호판 인식 후 서버로 `lot_id`, `plate_number` 전송
- `auto` 모드에서 입차/출차 자동 판정
- 카메라 가림 또는 번호판 이탈 후 재인식 준비 로그 출력

주차 현황 및 요금

- 홈 화면 활성 주차 세션 표시
- 기본요금과 추가요금 분리 표시
- 사전정산 후 미정산 금액 차감 및 출차 시 차액 자동 결제

6. 요금 정책

구간	요금	비고
입차 후 30분 이내	0원	무료 출차
31분~60분	기본요금 1,500원	현황판에 기본요금으로 유지 표시
61분부터	10분마다 500원	현황판에 추가요금으로 누적 표시
일일 최대	15,000원	여러 날이면 일 단위 최대요금을 합산

사전정산 후에는 **정산 완료 금액**이 올라가고 **현재 미정산 금액**이 내려갑니다. 출차 시에는 총요금에서 이미 정산한 금액을 뺀 차액만 결제합니다.

7. FastAPI 서버 사용법

서버 실행

```
cd /Users/jobyeongik/Documents/GitHub/Au-Park-project/services/api
.venv-conda/bin/python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

상태 확인

```
curl http://127.0.0.1:8000/health
```

API 문서

```
http://127.0.0.1:8000/docs
```

환경 변수

```
FIREBASE_DATABASE_URL=https://your-project-id-default-rtdb.firebaseio.com
FIREBASE_CREDENTIALS_PATH=/Users/jobyeongik/Downloads/firebase-service-account.json
```

```
GOOGLE_OAUTH_CLIENT_IDS=google-ios-client-id.apps.googleusercontent.com,google-android-cl  
PORTONE_API_SECRET=your-portone-secret
```

환경변수가 없으면 테스트/데모용 인메모리 저장소로 동작합니다.

8. Flutter 앱 사용법

로컬 시뮬레이터 실행

```
cd /Users/jobyeongik/Documents/GitHub/Au-Park-project/apps/mobile_flutter  
flutter run
```

실제 iPhone 릴리스 실행

```
cd /Users/jobyeongik/Documents/GitHub/Au-Park-project/apps/mobile_flutter  
flutter run --release \  
-d 00008110-0011743E2ED1801E \  
--dart-define=API_BASE_URL=https://YOUR-TUNNEL.trycloudflare.com/api/v1
```

Cloudflare 자동 실행 스크립트

```
cd /Users/jobyeongik/Documents/GitHub/Au-Park-project  
./scripts/run_mobile_cloudflare.sh -d 00008110-0011743E2ED1801E
```

실제 iPhone에서 `127.0.0.1` 은 MacBook이 아니라 iPhone 자기 자신입니다. 무선 테스트는 Cloudflare Tunnel URL 또는 같은 네트워크의 MacBook IP를 `API_BASE_URL` 로 넘겨야 합니다.

9. Vision Worker 사용법

설치

```
cd /Users/jobyeongik/Documents/GitHub/Au-Park-project/services/vision_worker
python3 -m venv .venv
.venv/bin/pip install -r requirements-ocr.txt
```

카메라 인덱스 확인

```
.venv/bin/python main.py scan-cameras --max-index 10
```

스냅샷 저장

```
.venv/bin/python main.py snapshot --source 0 --output data/plate_test.jpg
```

번호판 OCR 이미지 테스트

```
export SSL_CERT_FILE="$(.venv/bin/python -c 'import certifi; print(certifi.where())')"
.venv/bin/python main.py test-plate-image --config config.local.yaml --image data/plate_te
```

실시간 번호판 인식

```
export SSL_CERT_FILE="$(.venv/bin/python -c 'import certifi; print(certifi.where())')"
.venv/bin/python main.py plate --config config.local.yaml
```

주차면 점유 감지

```
.venv/bin/python main.py calibrate-occupancy --config config.local.yaml
.venv/bin/python main.py occupancy --config config.local.yaml
```

현재 번호판 인식은 OpenCV 후보 영역 검출과 EasyOCR을 사용합니다. 더 안정적인 운영 환경에서는 YOLO/ONNX 기반 번호판 검출기로 교체할 수 있습니다.

10. API 그룹

그룹	주요 엔드포인트	역할
auth	<code>/api/v1/auth/signup</code> , <code>/api/v1/auth/login</code> , <code>/api/v1/auth/social-login</code>	이메일/소셜 인증
users	<code>/api/v1/users/{user_id}</code>	사용자 조회 및 프로필
vehicles	<code>/api/v1/vehicles?user_id=...</code>	차량 등록/조회/삭제
parking	<code>/api/v1/parking/lots</code> , <code>/api/v1/parking/sessions/active</code>	주차장, 주차면, 활성 주차 세션, 요금 현황
payments	<code>/api/v1/payments/request</code> , <code>/api/v1/payments/methods</code>	사전정산, 자동정산 내역, 결제수단 등록
lpr	<code>/api/v1/lpr/auto</code> , <code>/api/v1/lpr/entry</code> , <code>/api/v1/lpr/exit</code>	번호판 인식 이벤트를 입출차 로직으로 연결
logs	<code>/api/v1/logs</code>	입출차 로그 조회

11. 핵심 파일

파일	설명
<code>apps/mobile_flutter/lib/core/api/api_client.dart</code>	Flutter 앱의 FastAPI 통신 클라이언트입니다.
<code>apps/mobile_flutter/lib/features/parking/screens/home_screen.dart</code>	활성 주차 세션, 기본요금, 추가요금, 정산완료, 미정산 금액을 표시하는 홈 현황판입니다.
<code>services/api/app/services/parking_service.py</code>	LPR 입차/출차, 활성 주차 세션, 사전정산 차감,

파일	설명
	자동정산을 처리합니다.
<code>services/api/app/services/parking_fee.py</code>	30분 무료, 기본요금, 추가요금, 일일 최대요금 정책을 계산합니다.
<code>services/api/app/services/repository.py</code>	Firebase Realtime Database 또는 InMemory 저장소를 추상화합니다.
<code>services/vision_worker/vision_worker/plate.py</code>	번호판 후보 검출, OCR, 중복 방지, auto 재무장 로직을 담당합니다.
<code>services/vision_worker/config.local.yaml</code>	카메라 source, LPR role, 프레임 간격, 반복 확인 횟수, ROI, OCR 설정을 관리합니다.
<code>scripts/run_mobile_cloudflare.sh</code>	FastAPI, Cloudflare Quick Tunnel, Flutter 릴리스 실행을 한 번에 연결합니다.

12. 운영 및 테스트 체크리스트

실제 iPhone 앱 테스트

1. FastAPI 서버 실행
2. Cloudflare Tunnel 실행 또는 자동 스크립트 사용
3. Flutter release 앱 설치
4. 회원가입/로그인
5. 차량번호와 결제수단 등록
6. Vision Worker 실행 후 번호판 인식

LPR auto 테스트

1. 번호판을 보여주면 입차 처리
2. 카메라를 손으로 막거나 번호판 없는 곳으로 이동
3. `LPR auto mode rearmed` 로그 확인
4. 번호판을 다시 보여주면 출차 처리
5. 남은 미정산 금액만 자동 결제

13. 현재 한계와 다음 단계

- Cloudflare Quick Tunnel은 재시작할 때 주소가 바뀌므로 장기 운영에는 named tunnel 또는 Cloud Run 같은 배포가 더 적합합니다.
- 현재 자동결제는 서버 결제 내역 생성 중심입니다. 실제 카드 승인/출금을 위해서는 PortOne 빌링키 결제 API 호출을 서버에 연결해야 합니다.
- EasyOCR 기반 번호판 인식은 테스트에 적합하지만, 운영 정확도를 높이려면 번호판 전용 YOLO/ONNX 검출 모델 도입이 좋습니다.
- 주차면 감지는 빈 주차장 기준 이미지 방식이 가능하며, 야외 환경에서는 YOLO 차량 검출 + 주차면 ROI 교차 판정 방식이 더 안정적입니다.
- 관리자 대시보드는 `apps/admin_dashboard` 에 자리만 마련되어 있으며 추후 구현 대상입니다.

Generated for Au-Park-project. Open this file directly in a browser to review the project structure and usage guide.